

图解大模型计算加速系列：FlashAttention V1，从硬件到计算逻辑

猛猿 公众号：大猿搬砖简记

1074 人赞同了该文章

大家好哇，好久没有更新了，今天想来讲讲Flash Attention（V1）。

不知道你有没有和我一样的感受，第一次读Flash Attention的论文时，感觉头懵懵的：它不仅涉及了硬件和cuda的知识，还涉及到很多计算逻辑上的trick。我的痛点是不能在头脑中具象化整个流程，就更不要提对细节的推导了。

所以这篇文章我读了很久，也写了很久（一个月），最终决定按照如下方式对Flash Attention进行介绍：

- 本文一到三部分，介绍相关硬件知识及Flash Attention诞生背景。
- 本文四到五部分，通过图解形式介绍forward/backward中的分块计算过程。所有的符号、公式都会给出详细的说明和推导过程。我在阅读中发现论文的一些推导不太符合直觉（or写得可能不太对），所以这里我在遵从论文符号表达的基础上，部分内容按自己的理解重新顺了一遍。
- 本文第六到第八部分，量化介绍Flash attention在性能上的改进，包括计算量、显存和IO复杂度。

【大模型计算加速系列】

猛猿：图解大模型计算加速系列：FlashAttention V1，从硬件到计算逻辑

猛猿：图解大模型计算加速系列：Flash Attention V2，从原理到并行计算

猛猿：图解Mixtral 8 * 7b推理优化原理与源码实现

猛猿：从啥也不会到CUDA GEMM优化

猛猿：图解大模型计算加速系列之：vLLM核心技术PagedAttention原理

猛猿：图解大模型计算加速系列：vLLM源码解析1，整体架构

猛猿：图解大模型计算加速系列：vLLM源码解析2，调度器策略(Scheduler)

猛猿：图解大模型计算加速系列：vLLM源码解析3，块管理器BlockManager（上篇）

猛猿：图解大模型计算加速系列：vLLM源码解析3、Prefix Caching（BlockManager下篇）



猛猿：图解大模型计算加速系列：分离式推理架构2，模糊分离与合并边界的chunked-prefills

【历史文章汇总】

猛猿：【必看】历史技术文章导航

一、Flash attention在做一件事

我们知道，对于Transformer类的模型，假设其输入序列长度为 N ，那么其计算复杂度和消耗的存储空间都为 $O(N^2)$ 。也就是说，随着输入序列的变长，将给计算和存储带来极大的压力。

因此，我们迫切需要一种办法，能解决Transformer模型的 $O(N^2)$ 复杂度问题。如果能降到 $O(N)$ ，那是最好的，即使做不到，逼近 $O(N)$ 那也是可以的。所以，Flash Attention就作为一种行之有效的解决方案出现了。

Flash Attention在做的事情，其实都包含在它的命名中了（Fast and Memory Efficient Exact Attention with IO-Awareness），我们逐一来看：

(1) Fast (with IO-Awareness)，计算快。在Flash Attention之前，也出现过一些加速Transformer计算的方法，这些方法的着眼点是“减少计算量FLOPs”，例如用一个稀疏attention做近似计算。但是Flash attention就不一样了，它并没有减少总的计算量，因为它发现：计算慢的卡点不在运算能力，而是在读写速度上。所以它通过降低对显存（HBM⁺）的访问次数来加快整体运算速度，这种方法又被称为IO-Awareness。在后文中，我们会详细来看Flash Attention是如何通过分块计算（tiling）和核函数融合（kernel fusion）来降低对显存的访问。

(2) Memory Efficient，节省显存。在标准attention场景中，forward时我们会计算并保存 $N \times N$ 大小的注意力矩阵；在backward时我们又会读取它做梯度计算，这就给硬件造成了 $O(N^2)$ 的存储压力。在Flash Attention中，则巧妙避开了这点，使得存储压力降至 $O(N)$ 。在后文中我们会详细看这个trick。

(3) Exact Attention，精准注意力。在（1）中我们说过，之前的办法会采用类似于“稀疏attention”的方法做近似。这样虽然能减少计算量，但算出来的结果并不完全等同于标准attention下的结果。但是Flash Attention却做到了完全等同于标准attention的实现方式，这也是后文我们讲述的要点。

二、计算限制与内存限制

在第一部分中我们提过，Flash Attention一个很重要的改进点是：由于它发现Transformer的计算瓶颈不在运算能力，而在读写速度上。因此它着手降低了对显存数据的访问次数，这才把整体计算效率提了上来。所以现在我们要问了：它是怎么知道卡点在读写速度上的？

为了解答这个问题，我们先来弄明白几个基本概念。

知乎 图解大模型计算加速系列：FlashAttention V1，从硬件到计算逻辑

[切换模式](#)

- β ：硬件带宽上限。指的是一个计算平台倾尽全力每秒所能完成的内存交换量。单位是Byte/s。
- π_t ：某个算法所需的总运算量，单位是FLOPs。下标 t 表示total。
- β_t ：某个算法所需的总数据读取存储量，单位是Byte。下标 t 表示total。

这里再强调一下对FLOPS和FLOPs的解释：

- FLOPS：等同于FLOP/s，表示Floating Point Operations Per Second，即每秒执行的浮点数据操作次数，用于衡量硬件计算性能。
- FLOPs：表示Floating Point Operations，表示某个算法的总计算量（即总浮点运算次数），用于衡量一个算法的复杂度。

我们知道，在执行运算的过程中，时间不仅花在计算本身上，也花在数据读取存储上，所以现在我们定义

- T_{cal} ：对某个算法而言，计算所耗费的时间，单位为s，下标cal表示calculate。其满足 $T_{cal} = \frac{\pi_t}{\pi}$
- T_{load} ：对某个算法而言，读取存储数据所耗费的时间，单位为s。其满足 $T_{load} = \frac{\beta_t}{\beta}$

我们知道，数据在读取的同时，可以计算；在计算的同时也可以读取，所以我们有：

- T ：对某个算法而言，完成整个计算所耗费的总时间，单位为s。其满足 $T = \max(T_{cal}, T_{load})$

也就是说，最终一个算法运行的总时间，取决于计算时间和数据读取时间中的最大值。

2.1 计算限制

当 $T_{cal} > T_{load}$ 时，算法运行的瓶颈在计算上，我们称这种情况为计算限制（math-bound）。此时我们有： $\frac{\pi_t}{\pi} > \frac{\beta_t}{\beta}$ ，即 $\frac{\pi_t}{\beta_t} > \frac{\pi}{\beta}$

2.2 内存限制

当 $T_{cal} < T_{load}$ 时，算法运行的瓶颈在数据读取上，我们称这种情况为内存限制（memory-bound）。此时我们有： $\frac{\pi_t}{\pi} < \frac{\beta_t}{\beta}$ ，即 $\frac{\pi_t}{\beta_t} < \frac{\pi}{\beta}$

我们称 $\frac{\pi_t}{\beta_t}$ 为算法的计算强度（Operational Intensity）

2.3 Attention计算中的计算与内存限制

本节内容参考自：[回旋托马斯x：FlashAttention:加速计算,节省显存, IO感知的精确注意力](#)

有了2.1和2.2的前置知识，现在我们可以来分析影响Transformer计算效率的因素到底是什么了。我们把目光聚焦到attention矩阵的计算上，其计算复杂度为 $O(N^2)$ ，是Transformer计算耗时的大头。

假设我们现在采用的硬件为A100-40GB SXM，同时采用混合精度训练（可理解为训练过程中的计算和存储都是fp16形式的，一个元素占用2byte）

我们要计算 $S = QK^T$ ，则有（对FLOPs要怎么算不了解的朋友，可以跳到6.1节进行阅读）：

$$\frac{\pi_t}{\beta_t} = \frac{2N^2d}{2Nd+2Nd+2N^2} = \frac{N^2d}{2Nd+N^2}$$

不同 N, d 取值下的受限类型如下：

N	d	ops/bytes	受限类型
256	64	43	<201, memory-bound
2048	64	60	<201, memory-bound
4096	64	62	<201, memory-bound
256	128	64	<201, memory-bound
2048	128	114	<201, memory-bound
4096	128	120	<201, memory-bound
256	256	85	<201, memory-bound
2048	256	205	>201, math-bound
4096	256	228	>201, math-bound

根据这个表格，我们可以来做下总结：

- 计算限制（math-bound）：大矩阵乘法（N和d都非常大）、通道数很大的卷积运算。相对而言，读得快，算得慢。
- 内存限制（memory-bound）：逐点运算操作。例如：激活函数、dropout、mask、softmax、BN和LN。相对而言，算得快，读得慢。

所以，我们第一部分中所说，“Transformer计算受限于数据读取”也不是绝对的，要综合硬件本身和模型大小来综合判断。但从表中的结果我们可知，memory-bound的情况还是普遍存在的，所以Flash attention的改进思想在很多场景下依然适用。

在Flash attention中，计算注意力矩阵时的softmax计算就受到了内存限制，这也是flash attention的重点优化对象，我们会在下文来详细看这一点。

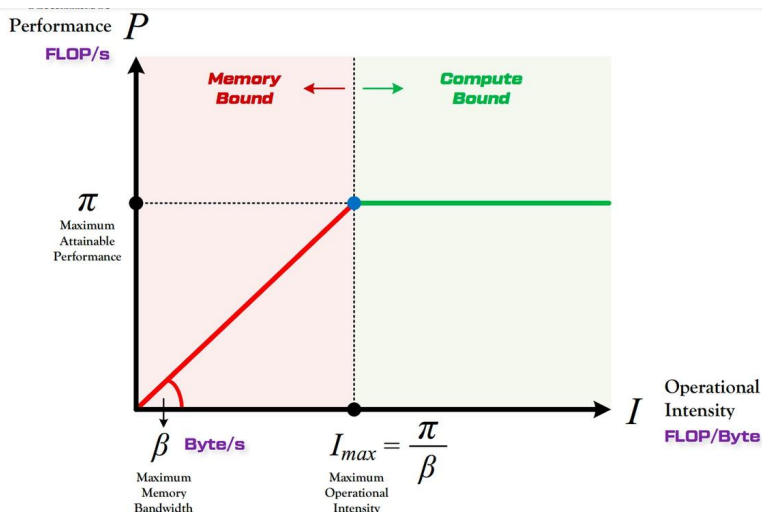
2.4 roof-line模型

其实到2.3为止，我们对计算限制和内存限制的概念已经知道得很清楚了。在这一节中，我们更系统来做一个总结。

一个算法运行的效率是离不开硬件本身的。我们往往想知道：对于一个运算量为 π_t ，数据读取存储量为 β_t 的算法，它在算力上限为 π ，带宽上限为 β 的硬件上，能达到的最大性能 P (Attanable Performance)是多少？

这里最大性能 P 指的是当前算法实际运行在硬件上时，每秒最多能达到的计算次数，单位是 FLOP/s 。

Roof-line模型+就是为了解答这一问题而提出的，它能直观帮我们看到算法在硬件上能跑得多快，模型见下图。



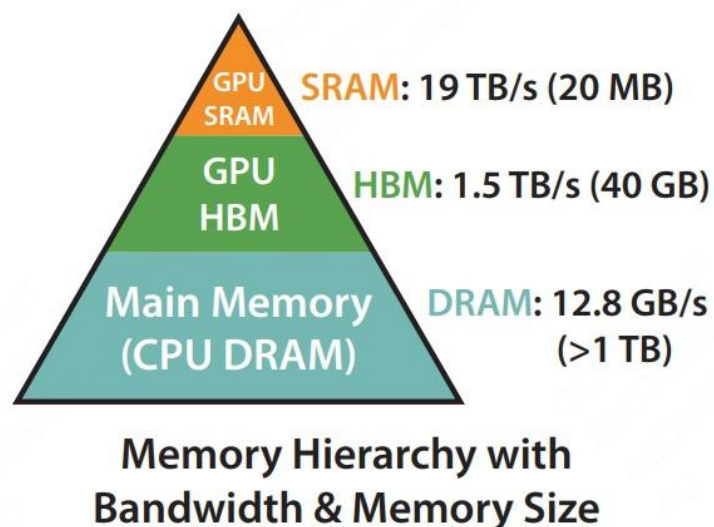
知乎 @猛猿

如图，横坐标 I 表示计算强度，满足 $I = \frac{\pi_i}{\beta_i}$ ；纵坐标 P 表示算法运行在硬件上的性能。算法的运行性能不会超过硬件本身的计算上限，所以 P 的最大值取到 π 。根据我们之前的分析，当 $I > \frac{\pi}{\beta}$ 时，存在计算限制；当 $I < \frac{\pi}{\beta}$ 时，存在内存限制。

三、GPU上的存储与计算

由于Flash attention的优化核心是减少数据读取的时间，而数据读取这块又离不开数据在硬件上的流转过程，所以这里我们简单介绍一些GPU上的存储与计算内容，作为Flash attention的背景知识。

3.1 GPU的存储分类



知乎 @猛猿

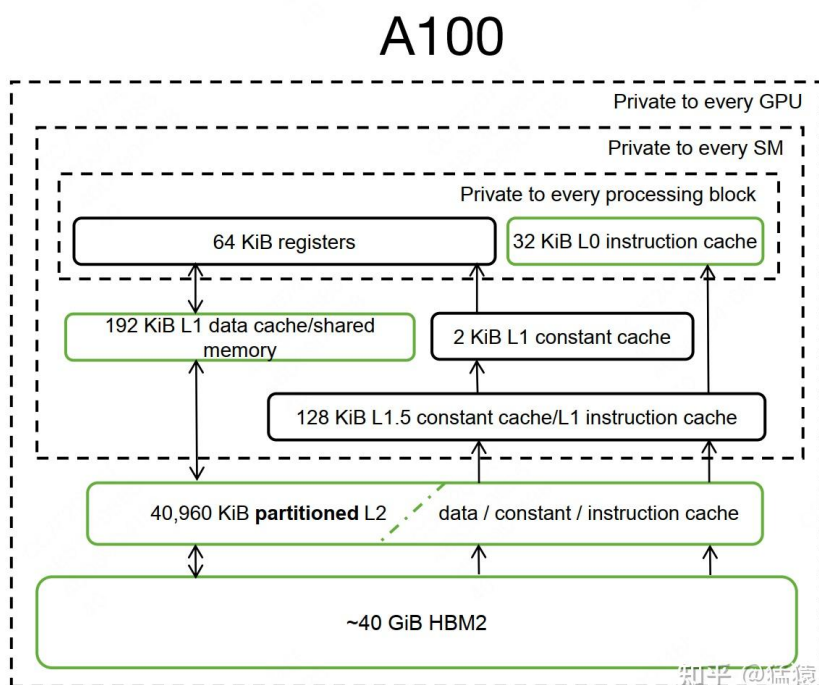
- 片上内存：主要用于缓存（cache）及少量特殊存储单元（例如texture），其特点是“存储空间小，但带宽大”。对应到上图中，SRAM+就属于片上内存，它的存储空间只有20MB，但是带宽可以达到19TB/s。
- 片下内存：主要用于全局存储（global memory），即我们常说的显存，其特点是“存储空间大，但带宽小”，对应到上图中，HBM就属于片下内存（也就是显存），它的存储空间有40GB（A100 40GB），但带宽相比于SRAM就小得多，只有1.5TB/s。

当硬件开始计算时，会先从显存（HBM）中把数据加载到片上（SRAM），在片上进行计算，然后将计算结果再写回显存中。那么这个“片上”具体长什么样，它又是怎么计算数据的呢？

3.2 GPU是如何做计算的



如图，负责GPU计算的一个核心组件叫SM（Streaming Multiprocessors+，流式多处理器），可以将其理解成GPU的计算单元，一个SM又可以由若干个SMP（SM Partition）组成，例如图中就由4个SMP组成。SM就好比CPU中的一个核，但不同的是一个CPU核一般运行一个线程，但是一个SM却可以运行多个轻量级线程（由Warp Scheduler控制，一个Warp



ref: <https://www.nvidia.com/en-us/on-demand/session/gtcspring21-s33322/>

- HBM2：即是我们的显存。
- L1缓存/shared memory：每个SM都有自己的L1缓存，用于存储SM内的数据，被SM内所有的cuda cores共享。SM间不能互相访问彼此的L1。NV Volta架构后，L1和shared memory合并（Volta架构前只有Kepler做过合并），目的是为了进一步降低延迟。合并过后，用户能写代码直接控制的依然是shared memory，同时可控制从L1中分配多少存储给shared memory。Flash attention中SRAM指的就是L1 cache/shared memory。
- L2缓存：所有SM共享L2缓存。L2缓存不直接由用户代码控制。L1/L2缓存的带宽都要比显存的带宽要大，也就是读写速度更快，但是它们的存储量更小。

现在再理一遍GPU的计算流程：将数据从显存（HBM）加载至on-chip的SRAM中，然后由SM读取并进行计算。计算结果再通过SRAM返回给显存。

我们知道显存的带宽相比SRAM要小的多，读一次数据是很费时的，但是SRAM存储又太小，装不下太多数据。所以我们就以SRAM的存储为上限，尽量保证每次加载数据都把SRAM给打满，节省数据读取时间。

3.3 kernel融合

前面说过，由于从显存读一次数据是耗时的，因此在SRAM存储容许的情况下，能合并的计算我们尽量合并在一起，避免重复从显存读取数据。

举例来说，我现在要做计算A和计算B。在老方法里，我做完A后得到一个中间结果，写回显存，然后再从显存中把这个结果加载到SRAM，做计算B。但是现在我发现SRAM完全有能力存下我的中间结果，那我就可以把A和B放在一起做了，这样就能节省很多读取时间，我们管这样的操作叫kernel融合。

由于篇幅限制，我们无法详细解释kernel这个概念，在这里大家可以粗犷地理解成是“函数”，它包含对线程结构（grid-block-thread）的定义，以及结构中具体计算逻辑的定义。理解到

kernel融合和尽可能利用起SRAM，以减少数据读取时间，都是flash attention的重要优化点。在后文对伪代码的解读中我们会看到，分块之后flash attention将矩阵乘法、mask、softmax、dropout操作合并成一个kernel，做到了只读一次和只写回一次，节省了数据读取时间。

好！目前为止所有的背景知识我们都介绍完了，现在我们直入主题，看看flash attention到底是怎么巧妙解决memory-bound问题。

四、Forward运作流程

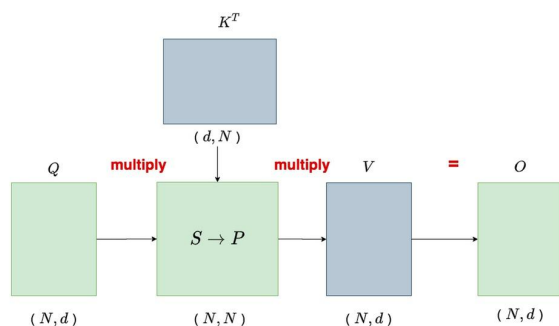
在后文相关的讲解中，我们遵循以下步骤：

- (1) 先看Flash Attention做分块计算的整体流程。
- (2) 再看分块的计算细节。
- (3) 最后看Flash Attention是如何通过分块计算控制I/O，进而解决memory-bound的问题，提升整体运算速度。

4.1 标准attention计算

这个大家应该都很熟悉了，假设一共有 N 个token，每个token向量的维度为 d ，则一个标准的attention计算如下图：

standard attention



知乎 @猛猿

其中， $S = QK^T$, $P = \text{softmax}(S)$ 。在GPT类的模型中，还需要对 P 做mask处理。为了表达方便，诸如mask、dropout之类的操作，我们都忽略掉，下文也是同理。

4.2 标准Safe softmax

这里我们需要额外强调 $P = \text{softmax}(S)$ 这一步。正常来说，假设 S 中某一行向量为 $[x_1, x_2, \dots, x_d]$ ，该行向量中的某一个元素为 x_i ，则对 S 做softmax后，有：

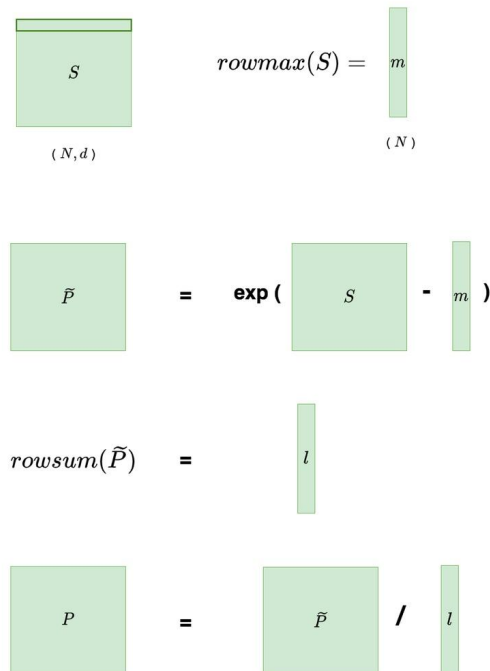
$$\text{softmax}(x_i) = \frac{e^{x_i}}{\sum_{j=1}^d e^{x_j}}$$

$$m(x) = \max_i x_i$$

$$\text{softmax}(x_i) = \frac{e^{x_i - m(x)}}{\sum_{j=1}^d e^{x_j - m(x)}}$$

下图展示了safe softmax的过程，这里 $\tilde{P}$, P 分别表示做归一化前和做归一化后的结果。大家记住图中 m, l 表达的含义，在后面的分块（Tiling）计算中，我们会用到这两个概念：

safe softmax

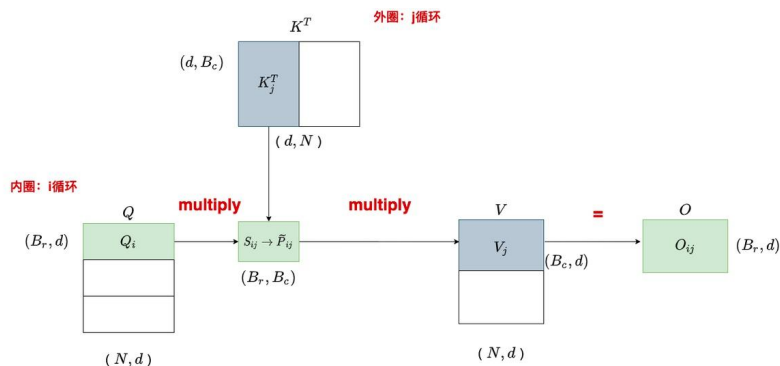


知乎 @猛猿

4.3 分块计算整体流程（Tiling）

我们知道Flash Attention的核心优化技术是采用了分块计算（Tiling），那么它是如何分块的？分块后的计算方式和不分块的计算方式又有哪些不同之处呢？

我们先来了解分块计算的整体流程（帮助大家理解数据块是怎么流转的），然后我们再针对其中的细节做一一讲解。



知乎 @猛猿

(2) 然后，将 K^T 矩阵切为 T_c 块，每块的长度为 B_c 。用 K_j^T 表示切完后的某块矩阵，则 K_j^T 的维度为 (d, B_c) 。易知 K_j^T 中存储着某 B_c 个token的key信息。

(3) 同样，将 V 矩阵也切为 T_c 块，每块长度为 B_c 。用 V_j 表示切完后的某块矩阵，则 V_j 的维度为 (B_c, d) 。易知 V_j 中存储着某 B_c 个token的value信息。

(4) 理解了上面的定义后，我们就可以开始做分块的attention计算了。以上图为例：

- 计算初始attention分数： $S_{ij} = Q_i * K_j^T = (B_r, d) * (d, B_c) = (B_r, B_c)$ ，图中的 S_{ij} 表示前 B_r 个token和前 B_c 个token间的原始相关性分数。
- Safe softmax + mask + dropout：对 S_{ij} 做safe softmax、mask和dropout操作，得到 $\tilde{P}_{ij}$ 。你可能会疑惑：前面不是说， $\tilde{P}_{ij}$ 是归一化前的结果， P_{ij} 是归一化后的结果吗？那么这里是不是应该用 P_{ij} 呢？这里确实只用算到 $\tilde{P}_{ij}$ ，在后文对分块计算细节的讲解中，我们会详细说这点。目前为止，大家不用太纠结符号，只用大体知道 P 代表的含义即可。
- 计算output： $O_{ij} = \tilde{P}_{ij} * V_j = (B_r, B_c) * (B_c, d) = (B_r, d)$ ，即可得到输出结果 O_{ij} 。细心的你肯定又发现了，这个等式不太对劲，这个 O_{ij} 不太对劲。想一想，在正常情况下，前 B_r 个token过attention后的输出结果，应该是它和所有token都做注意力计算后的输出结果。可是这里， O_{ij} 却只是前 B_r 个token和前 B_c 个token的结果。虽然 O_{ij} 的shape对了，但其中的内容却不是我们最终想要的。所以，关于 O 的计算，也是我们需要关注的细节，我们同样放在后文详说。

在计算这些分块时，GPU是可以做并行计算的，这也提升了计算效率。

好！现在你已经知道了单块的计算方式，现在让我们把整个流程流转起来把。在上图中，我们注明了 j 是外循环， i 是内循环，这个意思就是说，对于每个 j ，我们都把所有的 i 遍历一遍，得到相关结果。在论文里，又称为K，V是外循环，Q是内循环。写成代码就是：

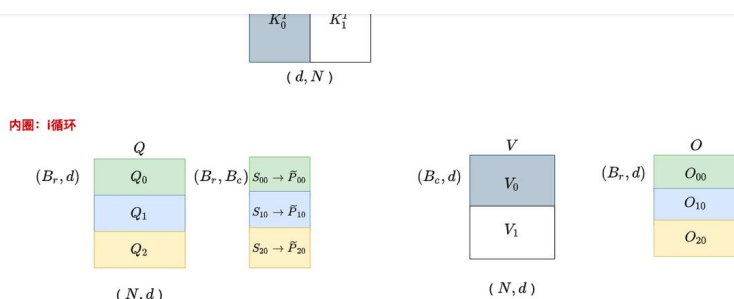
```
# -----
# Tc: K和V的分块数
# Tr: Q的分块数量
# -----
for 1 <= j <= Tc:
    for 1 <= i <= Tr:
        do....
```

如果你还有疑惑，那么下面两张图可以更直观地解答你的疑惑。

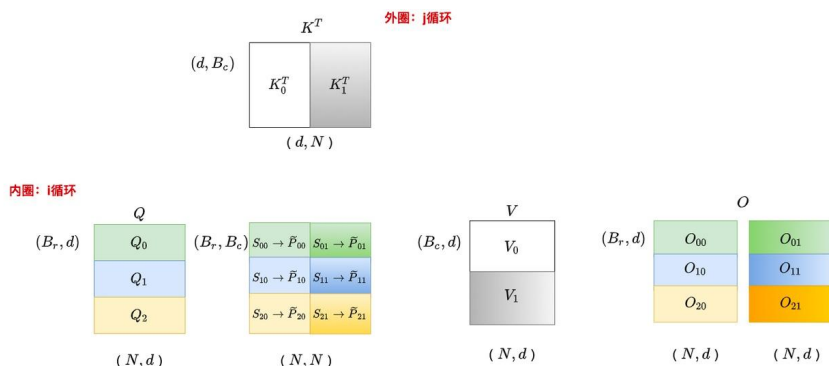
$j = 0$ ，遍历 i ：

知乎 图解大模型计算加速系列：FlashAttention V1，从硬件到计算逻辑

切换模式



知乎 @猛猿

 $j = 1$ ，遍历 i ：

知乎 @猛猿

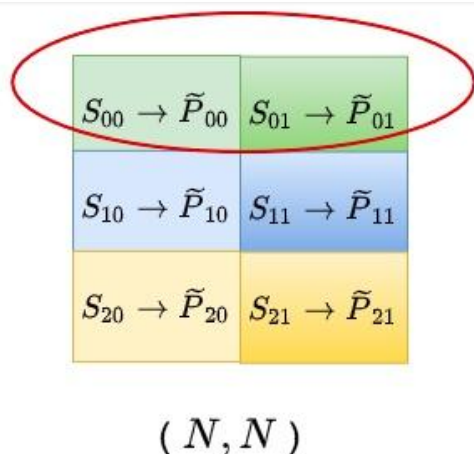
【△特别提醒】：正如上文所说，这里的 O 还需要经过一定的处理，才能和不分块场景下的 O 完全等价。这里我们将每一块的 O 单独画出，是为了帮助大家更好理解分块计算的整体流程，不代表它是最终的输出结果。

好！到这一步为止，我们已经掌握了使用Tiling计算attention的整体框架。但我们依然有很多细节问题没有解决：

- 分块后，要如何正确计算attention score？（即 S, P 的计算方法）
- 分块后，要如何正确计算输出 O ？
- 分块后，是如何实现优化I/O，解决memory-bound的问题的？

4.4 分块计算中的safe softmax

回顾之前绘制的安全safe softmax流程图，我们知道 m, l 都是针对完整的一行做rowmax、rowsum后的结果，那么在分块场景下，会变成什么呢？



知乎 @猛猿

以上图红圈内的数据为例，在标准场景下，我们是对红圈内的每一行做rowmax、rowsum后得到 $\tilde{P}$ 的。

现在切换到分块场景，我们分别算出了 S_{00} 和 S_{01} ，然后我们再对它们分别做rowmax、rowsum，是不是也能得到和标准场景下一模一样的结果呢？

答案当然是否定的。举个简单的例子，标准场景下的 $m(x)$ 是每行的全局最大值，可是分块后如果你也这么算，它就变成了局部最大值了。很明显，它不等同于标准场景下的结果。

所以，Flash Attention的作者们，在这里使用了一种巧妙的计算方式。

(1) 我们假设标准场景下， S 矩阵某一行的向量为 $x = [x_1, x_2, \dots, x_d]$ ，因为分块的原因，它被我们切成了两部分 $x = [x^{(1)}, x^{(2)}]$ 。

(2) 我们定义：

- $m(x)$ ：标准场景下，该行的全局最大值
- $m(x^{(1)})$ ：分块1的全局最大值
- $m(x^{(2)})$ ：分块2的全局最大值

那么易知： $m(x) = m([x^{(1)}, x^{(2)}]) = \max(m(x^{(1)}), m(x^{(2)}))$

(3) 我们定义：

- $f(x)$ ：标准场景下， $\exp(x - m(x))$ 的结果
- $f(x^{(1)})$ ：分块场景下， $\exp(x^{(1)} - m(x^{(1)}))$ 的结果
- $f(x^{(2)})$ ：分块场景下， $\exp(x^{(2)} - m(x^{(2)}))$ 的结果

那么易知： $f(x) = [e^{(m(x^{(1)}) - m(x))} f(x^{(1)}), e^{(m(x^{(2)}) - m(x))} f(x^{(2)})]$ 。这个很好理解，详细的证明过程就不写了。

(4) 我们定义：

- $l(x)$ ：标准场景下， $\text{rowsum}[f(x)]$ 的结果

收起

关于作者



猛猿

公众号：大猿搬砖简记

回答

66

文章

70

关注者

33,698

关注她

发私信

知乎 图解大模型计算加速系列：FlashAttention V1，从硬件到计算逻辑

切换模式

那么由 (3) 易知： $l(x) = e^{m(x^{(1)})-m(x)}l(x^{(1)}) + e^{m(x^{(2)})-m(x)}l(x^{(2)})$

(5) 现在，我们就可以用分块计算的结果，来表示标准场景下safe softmax的结果了：

$$\text{softmax}(x) = \frac{f(x)}{l(x)} = \frac{[e^{m(x^{(1)})-m(x)}f(x^{(1)}) + e^{m(x^{(2)})-m(x)}f(x^{(2)})]}{e^{m(x^{(1)})-m(x)}l(x^{(1)}) + e^{m(x^{(2)})-m(x)}l(x^{(2)})}$$

我们配合上面的图例和flash attention论文中的伪代码，再来进一步理解一下分块计算safe softmax的 (1) ~ (5) 步骤。

这里我们需注意：由于safe softmax是针对矩阵整行的计算，即相当于固定内圈 i ，移动外圈 j 的结果，所以在接下来的介绍中，我们都以这样的视角进行介绍。

Algorithm 1 FLASHATTENTION

Require: Matrices $\mathbf{Q}, \mathbf{K}, \mathbf{V} \in \mathbb{R}^{N \times d}$ in HBM, on-chip SRAM of size M .
 1: Set block sizes $B_c = \lfloor \frac{M}{4d} \rfloor$, $B_r = \min(\lfloor \frac{M}{4d} \rfloor, d)$.
 2: Initialize $\mathbf{O} = (0)_{N \times d} \in \mathbb{R}^{N \times d}$, $\ell = (0)_N \in \mathbb{R}^N$, $m = (-\infty)_N \in \mathbb{R}^N$ in HBM.
 3: Divide $\mathbf{Q}$ into $T_r = \lfloor \frac{N}{B_r} \rfloor$ blocks $\mathbf{Q}_1, \dots, \mathbf{Q}_{T_r}$ of size $B_r \times d$ each, and divide $\mathbf{K}, \mathbf{V}$ into $T_c = \lfloor \frac{N}{B_c} \rfloor$ blocks $\mathbf{K}_1, \dots, \mathbf{K}_{T_c}$ and $\mathbf{V}_1, \dots, \mathbf{V}_{T_c}$ of size $B_c \times d$ each.
 4: Divide $\mathbf{O}$ into T_r blocks $\mathbf{O}_1, \dots, \mathbf{O}_{T_r}$ of size $B_r \times d$ each, divide ℓ into T_r blocks $\ell_1, \dots, \ell_{T_r}$ of size B_r each, divide m into T_r blocks $m_1, \dots, m_{T_r}$ of size B_r each.
 5: **for** $1 \leq j \leq T_c$ **do**
 6: Load $\mathbf{K}_j, \mathbf{V}_j$ from HBM to on-chip SRAM.
 7: **for** $1 \leq i \leq T_r$ **do**
 8: Load $\mathbf{Q}_i, \mathbf{O}_i, \ell_i, m_i$ from HBM to on-chip SRAM.
 9: On chip, compute $\mathbf{S}_{ij} = \mathbf{Q}_i \mathbf{K}_j^T \in \mathbb{R}^{B_r \times B_c}$.
 10: On chip, compute $\tilde{m}_{ij} = \text{rowmax}(\mathbf{S}_{ij}) \in \mathbb{R}^{B_r}$, $\tilde{\mathbf{P}}_{ij} = \exp(\mathbf{S}_{ij} - \tilde{m}_{ij}) \in \mathbb{R}^{B_r \times B_c}$ (pointwise), $\tilde{\ell}_{ij} = \text{rowsum}(\tilde{\mathbf{P}}_{ij}) \in \mathbb{R}^{B_r}$.
 11: On chip, compute $m_i^{\text{new}} = \max(m_i, \tilde{m}_{ij}) \in \mathbb{R}^{B_r}$, $\ell_i^{\text{new}} = e^{m_i - m_i^{\text{new}}} \ell_i + e^{\tilde{m}_{ij} - m_i^{\text{new}}} \tilde{\ell}_{ij} \in \mathbb{R}^{B_r}$.
 12: Write $\mathbf{O}_i \leftarrow \text{diag}(\ell_i^{\text{new}})^{-1} (\text{diag}(\ell_i) e^{m_i - m_i^{\text{new}}} \mathbf{O}_i + e^{\tilde{m}_{ij} - m_i^{\text{new}}} \tilde{\mathbf{P}}_{ij} \mathbf{V}_j)$ to HBM.
 13: Write $\ell_i \leftarrow \ell_i^{\text{new}}$, $m_i \leftarrow m_i^{\text{new}}$ to HBM.
 14: **end for**
 15: **end for**
 16: Return $\mathbf{O}$.

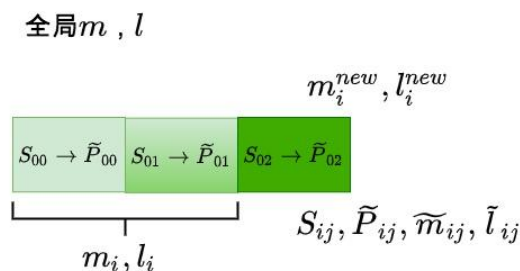
知乎 @猛猿

我们用 S_{00} （图中浅绿色方块）替换掉 (1) ~ (5) 步骤中的 $x^{(1)}$ ，用 S_{01} （图中深绿色方块）替换掉 $x^{(2)}$ 。我们关注点在伪代码部分的5-11行。

由于伪代码中的表达符太多，容易阻碍大家的理解，因此我们先明确各个数学符号表达的含义：

- S_{ij} ：对应在我们的例子里，就是 S_{00} 和 S_{01} ，即 $Q_i K_j^T$ 的结果
- $\tilde{m}_{ij}$ ：对于当前分块 S_{ij} 来说，每行的局部最大值。相当于前面步骤 (2) 中对 $m(x^{(1)}), m(x^{(2)})$ 的定义。
- $\tilde{\mathbf{P}}_{ij}$ ：分块场景下，各块的P矩阵（归一化前）结果。相当于步骤 (3) 中对 $f(x^{(1)}), f(x^{(2)})$ 的定义。
- $\tilde{\ell}_{ij}$ ：分块场景下，rowsum的结果。相当于步骤 (4) 中对 $l(x^{(1)}), l(x^{(2)})$ 的定义。
- m ：标准场景下，对 S 矩阵而言，每行的最大值，这是全局最大值（ m 首次定义在伪代码第2行），相当于前面步骤 (2) 中对 $m(x)$ 的定义
- l ：标准场景下，全局rowsum的结果（ l 首次定义在伪代码第2行），相当于前面步骤 (4) 中对 $l(x)$ 的定义。
- m_i ：表示 $\max(\tilde{m}_{i0}, \tilde{m}_{i1}, \dots, \tilde{m}_{i(j-1)})$ 。如果当前分块是 S_{ij} ，则 m_i 表示固定 i 时，前 $j-1$ 个分块中的局部最大值。容易推知，当固定 i ，遍历完 j 后， m_i 的结果就是全局最大值了。例如图例中，我们遍历完 S_{00}, S_{01} 后，就能得到全局最大值 m_0 。
- m_i^{new} ：表示 $\max(\tilde{m}_{i0}, \tilde{m}_{i1}, \dots, \tilde{m}_{i(j-1)}, \tilde{m}_{ij})$ 。如果当前分块是 S_{ij} ，则 m_i^{new} 表示固定 i 时，截止到当前分块为止的局部最大值。
- ℓ_i^{new} ：和 m_i^{new} 对应，相当于步骤 (4) 中用分块更新 $l(x)$ 的步骤。
- ℓ_i ：和 m_i 同理，即当我们将 j 遍历完后，我们就能得到针对 i 的全局rowmax和全局rowsum。而根据前面的定义， m^{new} 和 ℓ^{new} 是遍历完最新的 $S_{i:}$ 后得到的rowmax和rowsum。

- 所有以 ij 作为下标的，都表示当前分块的计算结果
- 所有以 i 作为下标的，都表示截止到前一个分块（包含前一个分块）的计算结果
- 所有以 new 为上标的，都表示引入当前分块做更新后的结果
- 所有没有下标的，都表示全局结果



知乎 @猛猿

相信通过上面对数学表发符的介绍，大家已经大致理解了分块计算safe softmax的过程，为了加深理解，现在我们来读一遍伪代码，把整个流程串起来：

- 伪代码第5~7行：从HBM（显存）上读取 K_j, V_j 到on-chip存储SRAM。注意，在代码处理逻辑上，这里是固定外圈 j ，循环内圈 i 。但是由于整个safe softmax逻辑是对“行”而言的，所以在理解时大家需要想像成固定内圈 i ，循环外圈 j ，也就是我们图例中绘制的深浅绿/蓝/黄色块。
- 伪代码第8行：从HBM（显存）上读取 Q_i, O_i, l_i, m_i 。记住我们之前说的trick，下标带 i 的都表示截止到前一个分块的计算结果。虽然我们前面没介绍过 O_i （在后文会细说），但按这个trick你应该也能猜到， O_i 也是随着分块的移动而逐步更新的。等移动到最后一个分块时，我们就能得到和标准场景下一模一样的输出结果 O_i 。在之前的图例中，为了方便大家对分块的整体流程有快速理解，我们画了很多个 O 出来，现在你应该能猜到，对每个 i ，我们只维持并不断更新一个 O_i ，直至遍历完毕（例如之前的图例中，我们画了6个 O ，但实际我们要维护更新的，只有3个： O_0, O_1, O_2 ）
- 伪代码第9行：正常计算 $S_{ij} = Q_i K_j^T$
- 伪代码第10行：基于当前分块 S_{ij} 计算 $\tilde{m}_{ij}, \tilde{P}_{ij}, \tilde{l}_{ij}$ 。
- 伪代码第11行：引入当前分块，计算截止目前为止的rowmax和rowsum，分别用 m_i^{new}, l_i^{new} 表示。
- 伪代码第12行：更新 O_i ，后文会详细解析这部分公式
- 伪代码第13行：用 m_i^{new}, l_i^{new} 去更新 m_i, l_i

讲完了分块safe softmax的伪代码，这时你可能发现一个问题了：之前你是否一直以为，在这一块操作后，分块计算得出的 $S, \tilde{P}$ 应该都和标准场景下的完全一致（比如应该是我们此前

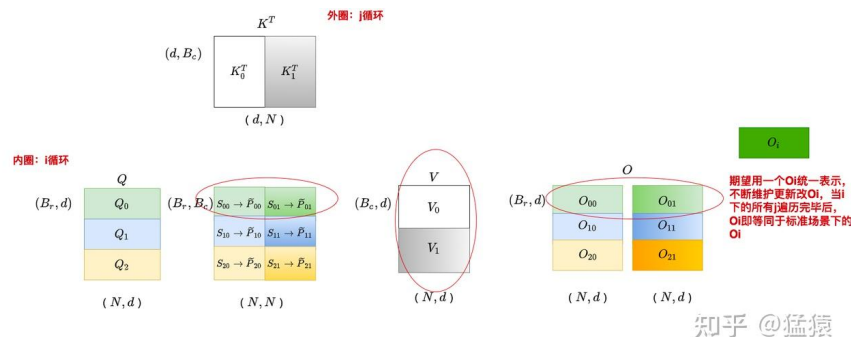
别急，还记得伪代码第12行我们说的更新 O_i 的公式么？分块计算的真正意义不在于得到正确的 $S, \tilde{P}$ ，而在于得到正确的 O 。

然后，你再来看伪代码5-13行，你会发现，在整个计算过程中，只有 m_i, l_i, O_i 被从on-chip的SRAM中写回到显存（HBM）中。把 i 都遍历完后，读写量也不过是 m, l, O 。相比于标准场景下，我们要读写的是 S, P, O ，读写量是不是一下就少很多，这不就能解决memory-bound的问题了吗。

所以，分块计算safe softmax的意义，就是抹去对 S, P 的读写。

4.5 分块计算中的输出O

终于到翘首以盼的输出 O 的分析部分了，当你第一次看到伪代码12行更新 $O_{\{i\}}$ 的公式，是不是觉得两眼一黑？不要紧，这里我们依然通过图解的方式，帮助大家理解并推导这个公式。



知乎 @猛猿

之前我们说过，上图中画的6个 O 并不是我们最终想要的结果。我们期望维护并更新 O_i ，当该 i 下的所有 j 遍历完毕后，我们的 O_i 就应该和标准场景下的 O_i 完全相等。

回到图例中，图中的 O_i 就应该等于被红框圈起来的 S, P 部分和 V 部分的乘积。但是别忘记之前说过，这里各块 S, P 都是局部rowmax，rowsum计算出来的结果。所以我们必须对各块 S, P 再做一些处理，才能让它们和 V 相乘，更新 O_i 。

那么要处理到什么程度为止呢？第一想法可能是，只要让每块 S, P 结果和标准场景下的结果完全一致，不就行了吗？但是别忘了，你不计算到最后一块 S, P ，你是拿不到全局的rowmax和rowsum的。而由于为了解决memory-bound的问题，我们只保留 m, l, O 而不存各块 S, P 。因此等你遍历到最后一块时，虽然有了全局的rowmax和rowsum，但没有 S, P ，你根本算不出最终的 O_i 。

所以这里我们换个思路： O_i 不是每遍历一块就更新一次吗？那有没有一种办法，**不断用当前最新的rowmax和rowsum去更新 O_i ，直到遍历完最后一块，这时的 O_i 不就标准场景下的结果完全一致了吗？也就是我们想构造形如下面这样的更新等式：**

$$O_i = O_i + \text{当前最新结果}$$

$$\begin{aligned}
& (j+1)^{-1} [\exp([S_{i,j}, S_{i(j+1)}] - m^{(j+1)})] \begin{bmatrix} V_{:,j} \\ V_{:,j+1} \end{bmatrix} \\
& (j+1)^{-1} [\exp(S_{i,j} - m^{(j+1)}) V_{:,j} + \exp(S_{i(j+1)} - m^{(j+1)}) V_{:,j+1}] \\
& (j+1)^{-1} [e^{-m^{(j+1)}} \exp(S_{i,j}) V_{:,j} + e^{-m^{(j+1)}} \exp(S_{i(j+1)}) V_{:,j+1}] \\
& (j+1)^{-1} [\text{diag}(l^{(j)}) e^{m^{(j)} - m^{(j+1)}} \text{diag}(l^{(j)})^{-1} \exp(S_{i,j} - m^{(j)}) V_{:,j} + e^{-m^{(j+1)}} \exp(S_{i(j+1)}) V_{:,j+1}] \\
& (j+1)^{-1} [\text{diag}(l^{(j)}) e^{m^{(j)} - m^{(j+1)}} P_{i,j} V_{:,j} + e^{-m^{(j+1)}} \exp(S_{i(j+1)}) V_{:,j+1}] \\
& (j+1)^{-1} [\text{diag}(l^{(j)}) e^{m^{(j)} - m^{(j+1)}} O_i^{(j)} + e^{-m^{(j+1)}} \exp(S_{i(j+1)} - \tilde{m}) V_{:,j+1}] \\
& (j+1)^{-1} [\text{diag}(l^{(j)}) e^{m^{(j)} - m^{(j+1)}} O_i^{(j)} + e^{-m^{(j+1)}} \tilde{P}_{i(j+1)} V_{:,j+1}]
\end{aligned}$$

初次看到这个推导过程，你可能有些懵圈，不要紧，我们一行一行来看。在讲解之前，我们先明确以上推导过程中符号上下标的含义：

- i ：这个大家应该很熟悉了。例如图例中， $i = 0, 1, 2$ 分别对应着深浅绿、深浅蓝、深浅黄块。
- $i(j+1)$ ：表示当前分块的相关结果
- $i, : j+1$ ：表示截止到当前分块（包含当前分块）的相关结果。 $i, : j$ 表示截止到前一分块（包含前一分块）的相关结果。

(1) 第一行：首先，我们期望的结果是，每遍历一个分块，就更新一次 O_i ，遍历完全部的分块后，我们就能得到和标准场景下完全一致的 O_i 。基于此我们有 $O_i^{(j+1)} = P_{i,j+1} V_{:,j+1}$ 。其中， $P_{i,j+1}$ 表示从第0个分块到当前分块，我们用当前最新的rowmax, rowsum更新一次所有分块的 P 结果（因为做过归一化了，所以是不带波浪号的 P ）。 $V_{:,j+1}$ 则表示当前分块及之前所有分块所对应的 V 部分（例如图例中，若当前分块是浅绿色块，则其对应着浅灰色 V ；若当前分块是深绿色块，则其对应着浅灰色+深灰色 V ）。

(2) 第二行：将 $P_{i,j+1}$ 改写成 $\text{softmax}(S_{i,j+1})$ 的形式。特别注意，这里 $S_{i,j+1}$ 所代表的各个分块间都是相互独立的，你可以理解为，只有在做 softmax 这个操作时，才考虑对这些独立的 S 用最新的rowmax, rowsum去更新 P 。

(3) 第三行：就是把(2)当中的 softmax 展开写了。即用当前最新的rowmax和rowsum去计算 P 。这里将 $S_{i,j+1}$ 拆成 $[S_{i,j}, S_{i(j+1)}]$ 两部分（[之前所有的分块，当前分块]）。同理拆 V 。

(4) ~ (5) 第四~五行：做简单的变式，不再赘述。

(6) 第六行：我们观察到，中括号式子里的前半部分，和之前所有分块的结果密切相关。联想到我们最终的目标是不断更新 O_i ，也就是在上一个 O_i 的基础上，引入当前分块的信息做更新。因此，能不能把上一个 O_i （对应到我们的式子里就是 $O_i^{(j)}$ ）表达出来呢？

基于这个思想做递推， $O_i^{(j)}$ 当然就是之前的所有分块，用上一分块的rowmax、rowsum做更新后求得 P ，再乘上对应的 V 得到的结果呀，所以根据此我们攒出了

$\text{diag}(l^{(j)})^{-1} \exp(S_{i,j} - m^{(j)}) V_{:,j}$ 这一项（就是 $O_i^{(j)}$ ），然后再用 $\text{diag}(l^{(j)}) e^{m^{(j)} - m^{(j+1)}}$ 去抵消我们在攒它的过程中引入的项。

(7)~(9)：第七~九行：明确了（6）以后，剩下的部分就很好理解啦。这里额外说下，为什么

现在再回头看伪代码的第12行，是不是就很清楚了呢？建议大家可以自行画图，动手推导，加深理解。

五、Backward运作流程

5.1 softmax求导

在后文对分块计算backward中，我们会频繁接触到和softmax求导相关的知识，繁杂的数学符号可能会使很多朋友看得蒙圈，所以这里我们做个快速复习。

设

$$\begin{cases} \mathbf{y} = \text{softmax}(\mathbf{z}) \\ L = f(\mathbf{y}) \end{cases}$$

其中， L 表示Loss， $f(\cdot)$ 表示Loss函数， $\mathbf{y} = [y_1 \ y_2 \ y_3]$ ， $\mathbf{z} = [z_1 \ z_2 \ z_3]$ ，若现在想求 $\frac{\partial L}{\partial z_j}$ ，要怎么算呢？

根据链式法则，我们有 $\frac{\partial L}{\partial z_j} = \frac{\partial L}{\partial y} \frac{\partial y}{\partial z_j}$ ，所以我们分别来看这两项。

$$(1) \frac{\partial L}{\partial y}$$

我们现在不考虑具体的Loss函数，直接假设这一项的结果为 $[m_1 \ m_2 \ m_3]$

$$(2) \frac{\partial y}{\partial z_j}$$

我们知道，对于某个 z_j 来说，在softmax的操作下，它参与了 y_1, y_2, y_3 三者的计算，因此它的偏导也和这三者密切相关，这里我们分成两种情况：

$$\begin{cases} \frac{\partial y_i}{\partial z_j} = y_i(1 - y_i), \text{ 当 } i = j \\ \frac{\partial y_i}{\partial z_j} = -y_i y_j, \text{ 当 } i \neq j \end{cases}$$

这里不再赘述详细的推动过程，有需要的朋友可以参考[这篇文章](#)。

有了这个理解，我们再来谈谈基于 $\mathbf{y} = \text{softmax}(\mathbf{z})$ 的Jacobian矩阵 $\text{diag}(\mathbf{y}) - \mathbf{y}^T \mathbf{y}$ ：

$$\begin{aligned} \text{diag}(\mathbf{y}) - \mathbf{y}^T \mathbf{y} &= \begin{bmatrix} y_1 & 0 & 0 \\ 0 & y_2 & 0 \\ 0 & 0 & y_3 \end{bmatrix} - \begin{bmatrix} y_1 \\ y_2 \\ y_3 \end{bmatrix} * \begin{bmatrix} y_1 & y_2 & y_3 \end{bmatrix} \\ &= \begin{bmatrix} y_1 - y_1^2 & -y_1 y_2 & -y_1 y_3 \\ -y_2 y_1 & y_2 - y_2^2 & -y_2 y_3 \\ -y_3 y_1 & -y_3 y_2 & y_3 - y_3^2 \end{bmatrix} \end{aligned}$$

$$(3) \frac{\partial L}{\partial z_j} = \frac{\partial L}{\partial y} \frac{\partial y}{\partial z_j}$$

有了 (1) (2) 的结果，现在就可以来推导 $\frac{\partial L}{\partial z_j}$ ，我们有：

$$\frac{\partial L}{\partial z_j} = \frac{\partial L}{\partial y} \frac{\partial y}{\partial z_j} = \sum_{i=1}^I \frac{\partial L}{\partial y_i} \frac{\partial y_i}{\partial z_j} = y_j (dy_j - \sum_{j=1}^I y_j dy_j)$$

举个例子，若我们现在想求 $\frac{\partial L}{\partial z_1}$ ，我们将 $\frac{\partial L}{\partial y} = [m_1 \quad m_2 \quad m_3]$ 代入上面公式，则有：

$$\frac{\partial L}{\partial z_1} = m_1(y_1 - y_1^2) - m_2 y_1 y_2 - m_3 y_1 y_3$$

现在，针对所有的 z ，我们将 $\frac{\partial L}{\partial z}$ 写成矩阵表达式有：

$$\begin{aligned} \frac{\partial L}{\partial z} &= \frac{\partial L}{\partial y} \frac{\partial y}{\partial z} = dy(diag(y) - y^T y) \\ &= [m_1 \quad m_2 \quad m_3] \left(\begin{bmatrix} y_1 & 0 & 0 \\ 0 & y_2 & 0 \\ 0 & 0 & y_3 \end{bmatrix} - \begin{bmatrix} y_1 \\ y_2 \\ y_3 \end{bmatrix} \begin{bmatrix} y_1 & y_2 & y_3 \end{bmatrix} \right) \\ &= [m_1 \quad m_2 \quad m_3] \begin{bmatrix} y_1 - y_1^2 & -y_1 y_2 & -y_1 y_3 \\ -y_2 y_1 & y_2 - y_2^2 & -y_2 y_3 \\ -y_3 y_1 & -y_3 y_2 & y_3 - y_3^2 \end{bmatrix} \end{aligned}$$

至此，大家记住这两个重要的结论：

$$\frac{\partial L}{\partial z} = \frac{\partial L}{\partial y} \frac{\partial y}{\partial z} = dy(diag(y) - y^T y)$$

$$\frac{\partial L}{\partial z_j} = y_j (dy_j - \sum_{j=1}^I y_j dy_j)$$

5.2 标准backward计算

Algorithm 3 Standard Attention Backward Pass

Require: Matrices $\mathbf{Q}, \mathbf{K}, \mathbf{V}, \mathbf{dO} \in \mathbb{R}^{N \times d}$, $\mathbf{P} \in \mathbb{R}^{N \times N}$ in HBM.

- 1: Load $\mathbf{P}, \mathbf{dO}$ by blocks from HBM, compute $\mathbf{dV} = \mathbf{P}^T \mathbf{dO} \in \mathbb{R}^{N \times d}$, write $\mathbf{dV}$ to HBM.
- 2: Load $\mathbf{dO}, \mathbf{V}$ by blocks from HBM, compute $\mathbf{dP} = \mathbf{dO} \mathbf{V}^T \in \mathbb{R}^{N \times N}$, write $\mathbf{dP}$ to HBM.
- 3: Read $\mathbf{P}, \mathbf{dP}$ from HBM, compute $\mathbf{dS} \in \mathbb{R}^{N \times N}$ where $dS_{ij} = P_{ij}(dP_{ij} - \sum_l P_{il} dP_{il})$, write $\mathbf{dS}$ to HBM.
- 4: Load $\mathbf{dS}$ and $\mathbf{K}$ by blocks from HBM, compute $\mathbf{dQ} = \mathbf{dS} \mathbf{K}$, write $\mathbf{dQ}$ to HBM.
- 5: Load $\mathbf{dS}$ and $\mathbf{Q}$ by blocks from HBM, compute $\mathbf{dK} = \mathbf{dS}^T \mathbf{Q}$, write $\mathbf{dK}$ to HBM.
- 6: Return $\mathbf{dQ}, \mathbf{dK}, \mathbf{dV}$.

知乎 @猛猿

我们先来总结下forward中做的操作，为了表达简便，这里将mask、dropout等零碎操作省去，同时假设 $f(\cdot)$ 是损失函数：

$$\begin{aligned} S &= \mathbf{Q} \mathbf{K}^T \\ P &= \text{softmax}(S) \\ O &= \mathbf{P} \mathbf{V} \\ L &= f(O) \end{aligned}$$

5.3 分块backward计算

在讲解backward计算前，我们先来看看经过分块Forward计算后，显存（HBM）上都存了哪些数据：

- m ：全局rowmax
- l ：全局rowsum
- Q, K, V ：等同于标准attention场景下的结果
- O ：等同于标准attention场景下的输出结果
- dO ：有了完整的 O ，我们就可以按正常的backward步骤先求出它的梯度，也存放在显存上。然后我们就能按照链式法则，分块地去求别的矩阵的梯度了。

既然有了全局的 m, l ，那么现在对于任意一块 S_{ij} ，我们就能基于 m, l 算出和标准场景下完全一致的 P_{ij} 了。因此，在backward的过程中，flash attention将采用重计算的方式，重新算出 S_{ij}, P_{ij} ，并将它们运用到backward的计算中去，所以在接下来的讲解中，大家就可以把 S, P 理解成完全等同于标准场景下的结果，而不是像分块计算forward中那样的 S, P 。

另外需要注意的是，为了简化表达，在接下来的分析中，关于mask、dropout之类的步骤，我们在表述上都略去。现在让我们来看分块计算backward的伪代码：

Algorithm 4 FLASHATTENTION Backward Pass

Require: Matrices $Q, K, V, O, dO \in \mathbb{R}^{N \times d}$ in HBM, vectors $\ell, m \in \mathbb{R}^N$ in HBM, on-chip SRAM of size M , softmax scaling constant $\tau \in \mathbb{R}$, masking function MASK, dropout probability p_{drop} , pseudo-random number generator state $\mathcal{R}$ from the forward pass.

- 1: Set the pseudo-random number generator state to $\mathcal{R}$.
- 2: Set block sizes $B_c = \lceil \frac{M}{4d} \rceil$, $B_r = \min(\lceil \frac{M}{4d} \rceil, d)$.
- 3: Divide Q into $T_r = \lceil \frac{N}{B_r} \rceil$ blocks $Q_1, \dots, Q_{T_r}$ of size $B_r \times d$ each, and divide K, V in to $T_c = \lceil \frac{N}{B_c} \rceil$ blocks $K_1, \dots, K_{T_c}$ and $V_1, \dots, V_{T_c}$ of size $B_c \times d$ each.
- 4: Divide O into T_r blocks $O_1, \dots, O_{T_r}$ of size $B_r \times d$ each, divide dO into T_r blocks $dO_1, \dots, dO_{T_r}$ of size $B_r \times d$ each, divide ℓ into T_r blocks $\ell_1, \dots, \ell_{T_r}$ of size B_r each, divide m into T_r blocks $m_1, \dots, m_{T_r}$ of size B_r each.
- 5: Initialize $dQ = (0)_{N \times d}$ in HBM and divide it into T_r blocks $dQ_1, \dots, dQ_{T_r}$ of size $B_r \times d$ each. Initialize $dK = (0)_{N \times d}$, $dV = (0)_{N \times d}$ in HBM and divide dK, dV in to T_c blocks $dK_1, \dots, dK_{T_c}$ and $dV_1, \dots, dV_{T_c}$ of size $B_c \times d$ each.
- 6: **for** $1 \leq j \leq T_c$ **do**
- 7: Load K_j, V_j from HBM to on-chip SRAM.
- 8: Initialize $dK_j = (0)_{B_r \times d}$, $dV_j = (0)_{B_c \times d}$ on SRAM.
- 9: **for** $1 \leq i \leq T_r$ **do**
- 10: Load $Q_i, O_i, dO_i, dQ_i, \ell_i, m_i$ from HBM to on-chip SRAM.
- 11: On chip, compute $S_{ij} = \tau Q_i K_j^T \in \mathbb{R}^{B_r \times B_c}$.
- 12: On chip, compute $S_{ij}^{\text{masked}} = \text{MASK}(S_{ij})$.
- 13: On chip, compute $P_{ij} = \text{diag}(\ell_i)^{-1} \exp(S_{ij}^{\text{masked}} - m_i) \in \mathbb{R}^{B_r \times B_c}$.
- 14: On chip, compute dropout mask $Z_{ij} \in \mathbb{R}^{B_r \times B_c}$ where each entry has value $\frac{1}{1-p_{\text{drop}}}$ with probability $1-p_{\text{drop}}$ and value 0 with probability p_{drop} .
- 15: On chip, compute $P_{ij}^{\text{dropped}} = P_{ij} \circ Z_{ij}$ (pointwise multiply).
- 16: On chip, compute $dV_j \leftarrow dV_j + (P_{ij}^{\text{dropped}})^T dO_i \in \mathbb{R}^{B_c \times d}$.
- 17: On chip, compute $dP_{ij}^{\text{dropped}} = dO_i V_j^T \in \mathbb{R}^{B_r \times B_c}$.
- 18: On chip, compute $dP_{ij} = dP_{ij}^{\text{dropped}} \circ Z_{ij}$ (pointwise multiply).
- 19: On chip, compute $D_i = \text{rowsum}(dO_i \circ O_i) \in \mathbb{R}^{B_r}$.
- 20: On chip, compute $dS_{ij} = P_{ij} \circ (dP_{ij} - D_i) \in \mathbb{R}^{B_r \times B_c}$.
- 21: Write $dQ_i \leftarrow dQ_i + \tau dS_{ij} K_j \in \mathbb{R}^{B_r \times d}$ to HBM.
- 22: On chip, compute $dK_j \leftarrow dK_j + \tau dS_{ij}^T Q_i \in \mathbb{R}^{B_c \times d}$.
- 23: **end for**
- 24: Write $dK_j \leftarrow dK_j$, $dV_j \leftarrow dV_j$ to HBM.
- 25: **end for**
- 26: Return dQ, dK, dV .

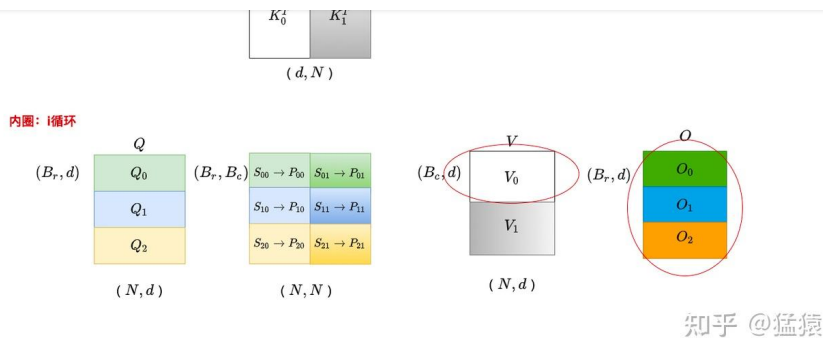
知乎 @猛猿

(1) 求 V_j 梯度

由Forward过程我们知： $O = PV$ ，因此有了 dO 后，我们就可以先来求 dP 和 dV 了。观察下方的图，我们会发现此时所有的 P 都是不带波浪号的，再强调一下，这是因为经过了重计算。此外 S, P 的结果都等同于标准场景下的结果，而不是forward中所代表的含义。

知乎 图解大模型计算加速系列：FlashAttention V1，从硬件到计算逻辑

切换模式



假设现在 $j = 0$ ，那我们要怎么求 dV_0 呢？

我们先来看 V_0 都参与了 O 哪些部分的计算，以及是怎么参与的：由图可知， P_{00} 和 V_{00} 参与了 O_0 的计算， P_{10} 和 V_{00} 参与了 O_1 的计算， P_{20} 和 V_0 参与了 O_2 的计算。所以我们有：

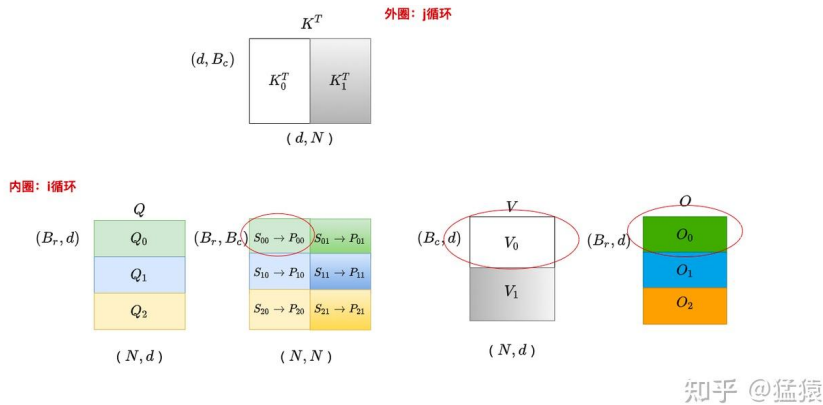
$$dV_0 = (P_{00})^T dO_0 + (P_{10})^T dO_1 + (P_{20})^T dO_2$$

进而推知：

$$dV_j = \sum_i (P_{ij})^T dO_i$$

在伪代码11~15行中，做的都是 S, P 重计算的过程，伪代码的第16行，就是在按这个方法分块计算并累积 dV_j 。

(2) 求 P_{ij} 梯度



观察上图，可以发现 P_{ij} 只与 V_j, O_i 相关，例如 P_{10} 只与 V_0, O_1 相关。因此我们有：

$$dP_{ij} = dO_i V_j^T$$

这就是伪代码第17行做的事情。

(3) 求 S_{ij} 梯度

这一块是令许多人感到迷惑的，我们先来回顾下“softmax求导”部分让大家记住的一个重要结论：

我们假设 s_i, p_i, o_i 分别为矩阵 S, P, O 的某一行（注意这里 i 不是表示第 i 块的意思，是表示第 i 行，所以我们用小写的 s, p, o 表示），那么根据这个结论，我们有：

$$\begin{aligned} ds_i &= dp_i(diag(p_i) - p_i^T p_i) \\ &= dp_i diag(p_i) - dp_i p_i^T p_i \\ &= dp_i diag(p_i) - do_i V^T p_i^T p_i \\ &= dp_i diag(p_i) - do_i o_i^T p_i \\ &= p_i \circ [dp_i - rowsum(do_i \circ o_i)] \end{aligned}$$

你可能对这个推导的最后一步有疑惑：为什么要大费周章，将 ds_i 改写成这么复杂的形式呢？因为在最后一步之前，我们都是针对“某一行”来求导，而引入最后一步的目的，是为了延展至对“某一块（多行）”的求导，也就是说针对某一块 dS_i （注意这里是大写的 S ， i 的含义也回归至“第几块”），我们有：

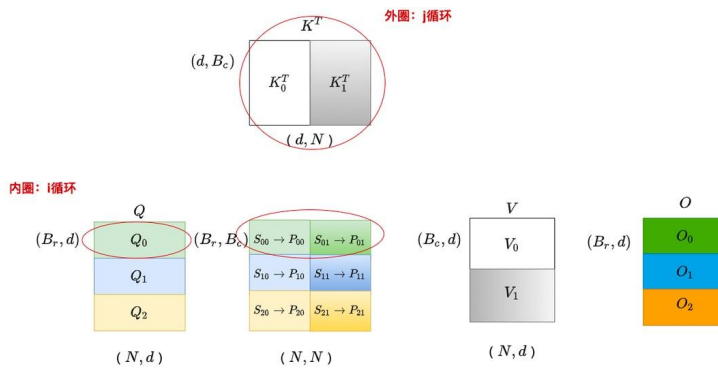
$$dS_i = P_i \circ [dP_i - rowsum(dO_i \circ O_i)]$$

如果实在难以理解推导过程，建议大家带一些具体的值进去，就能理解我们为什么要写成这种形式了。进而，我们可以推知：

$$dS_{ij} = P_{ij} \circ [dP_{ij} - rowsum(dO_i \circ O_i)]$$

这就是伪代码第19~20行做的事情。

(4) 求 Q_i 梯度



知乎 @猛猿

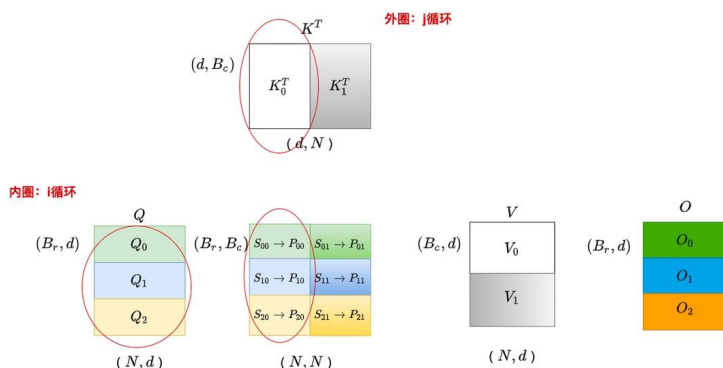
到目前为止，我们已经知道 dS_{ij} ，那么现在就可以根据链式法则继续求 dQ_i 了。

对照上图，我们把目光聚焦在 Q_0 身上，由forward过程可知：

$$\begin{aligned} S_{00} &= Q_0 K_0^T \\ S_{01} &= Q_0 K_1^T \end{aligned}$$

因此，针对 Q_0 ，我们有： $dQ_0 = dS_{00} K_0 + dS_{01} K_1$

推广到任意 Q_i ，我们有： $dQ_i = \sum_j dS_{ij} K_j$

(5) 求 K_j 梯度

知乎 @猛猿

这一步就很简单啦，如果你被复杂的分块推导弄懵了脑袋，那不妨再复习一下我们前面提过的trick：对照上图，取出某一块 K_j 。由于我们是从 dS_{ij} 链式推向 K_j ，所以这里只要搞明白这块 K_j 和哪些 Q 一起计算出了哪些 S 再把相关结果相加即可。

只要看了流程图，就不难得知：某块 K_j 和对应的 Q_i 共同计算出了对应的 S_{ij} ，因此有：

$$dK_j = \sum_i dS_{ij}^T Q_i$$

这就是伪代码第22行做的事情。

好！现在我们就把分块backward的细节讲完了，当大家感到迷茫时，一定记得画图；在碰到需要做累加才能计算出梯度的步骤中，画图也可以帮助我们快速理解是按 i 维度还是按 j 维度进行累加。

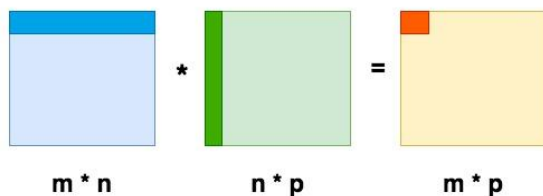
六、计算量和显存需求

6.1 矩阵相乘的计算量

我们先来看一个前置知识：两个矩阵相乘，要怎么统计它们的计算量？

我们一般用FLOPs（floating point operations，浮点运算次数）来表示运算量的大小。对于“两矩阵相乘”这个操作而言，其运算量 = 乘法运算的次数 + 加法运算的次数。

来看一个具体例子：



知乎 @猛猿

那么现在结果矩阵中，一共有 $m \times p$ 个橘色方块，则意味着我们需要进行： $m \times p \times (n + n - 1)$ 次浮点计算。

再进一步，假设此时在蓝色和绿色的矩阵外，我们还有一个**bias**矩阵，意味着计算单个橘色方块时我们需要进行 n 次乘法和 $n-1+1$ 次加法运算，那么此时总计算量为： $m \times p \times (n+n) = 2mnp$ 。当然，即使不加这个**bias**，我们也可以把-1项给忽略，得到相同的结果。

所以这里我们总结下，假设有两个矩阵A和B，它们的维度分别为 (m, n) 和 (n, p) ，则这两矩阵相乘的运算量为 $2mnp$ 。

一般在矩阵运算中，乘法运算的时间要高于加法运算的时间，因此有时在统计运算量时，我们只考虑乘法运算的次数，则此时两矩阵相乘的运算量可近似为 mnp

6.2 Flash Attention的计算量

有了前置知识，我们就能分析flash attention的计算量了，我们以forward过程为例（为了大家阅读方便，我们再把forward的伪代码放一遍）：

Algorithm 1 FLASHATTENTION

Require: Matrices $\mathbf{Q}, \mathbf{K}, \mathbf{V} \in \mathbb{R}^{N \times d}$ in HBM, on-chip SRAM of size M .

- 1: Set block sizes $B_c = \lceil \frac{M}{4d} \rceil, B_r = \min(\lceil \frac{M}{4d} \rceil, d)$.
- 2: Initialize $\mathbf{O} = (0)_{N \times d} \in \mathbb{R}^{N \times d}, \ell = (0)_N \in \mathbb{R}^N, m = (-\infty)_N \in \mathbb{R}^N$ in HBM.
- 3: Divide $\mathbf{Q}$ into $T_r = \lceil \frac{N}{B_r} \rceil$ blocks $\mathbf{Q}_1, \dots, \mathbf{Q}_{T_r}$ of size $B_r \times d$ each, and divide $\mathbf{K}, \mathbf{V}$ in to $T_c = \lceil \frac{N}{B_c} \rceil$ blocks $\mathbf{K}_1, \dots, \mathbf{K}_{T_c}$ and $\mathbf{V}_1, \dots, \mathbf{V}_{T_c}$ of size $B_c \times d$ each.
- 4: Divide $\mathbf{O}$ into T_r blocks $\mathbf{O}_1, \dots, \mathbf{O}_{T_r}$ of size $B_r \times d$ each, divide ℓ into T_r blocks $\ell_1, \dots, \ell_{T_r}$ of size B_r each, divide m into T_r blocks $m_1, \dots, m_{T_r}$ of size B_r each.
- 5: **for** $1 \leq j \leq T_c$ **do**
- 6: Load $\mathbf{K}_j, \mathbf{V}_j$ from HBM to on-chip SRAM.
- 7: **for** $1 \leq i \leq T_r$ **do**
- 8: Load $\mathbf{Q}_i, \mathbf{O}_i, \ell_i, m_i$ from HBM to on-chip SRAM.
- 9: On chip, compute $\mathbf{S}_{ij} = \mathbf{Q}_i \mathbf{K}_j^T \in \mathbb{R}^{B_r \times B_c}$.
- 10: On chip, compute $\tilde{m}_{ij} = \text{rowmax}(\mathbf{S}_{ij}) \in \mathbb{R}^{B_r}, \tilde{\mathbf{P}}_{ij} = \exp(\mathbf{S}_{ij} - \tilde{m}_{ij}) \in \mathbb{R}^{B_r \times B_c}$ (pointwise), $\tilde{\ell}_{ij} = \text{rowsum}(\tilde{\mathbf{P}}_{ij}) \in \mathbb{R}^{B_r}$.
- 11: On chip, compute $m_i^{\text{new}} = \max(m_i, \tilde{m}_{ij}) \in \mathbb{R}^{B_r}, \ell_i^{\text{new}} = e^{m_i - m_i^{\text{new}}} \ell_i + e^{\tilde{m}_{ij} - m_i^{\text{new}}} \tilde{\ell}_{ij} \in \mathbb{R}^{B_r}$.
- 12: Write $\mathbf{O}_i \leftarrow \text{diag}(\ell_i^{\text{new}})^{-1} (\text{diag}(\ell_i) e^{m_i - m_i^{\text{new}}} \mathbf{O}_i + e^{\tilde{m}_{ij} - m_i^{\text{new}}} \tilde{\mathbf{P}}_{ij} \mathbf{V}_j)$ to HBM.
- 13: Write $\ell_i \leftarrow \ell_i^{\text{new}}, m_i \leftarrow m_i^{\text{new}}$ to HBM.
- 14: **end for**
- 15: **end for**
- 16: Return $\mathbf{O}$.

知乎 @猛猿

我们知道矩阵相乘运算占据了运算量的大头，因此我们把分析目光集中到所有的矩阵运算上来。

(1) 在代码第9行，我们有 $\mathbf{S}_{ij} = \mathbf{Q}_i \mathbf{K}_j^T$ ，其中 $\mathbf{Q}_i \in \mathbb{R}^{B_r \times d}, \mathbf{K}_j^T \in \mathbb{R}^{d \times B_c}$ 。根据前置知识，求 $\mathbf{S}_{ij}$ 的计算量为 $O(B_r B_c d)$ 。

(2) 在代码第12行，我们有 $\tilde{\mathbf{P}}_{ij} \mathbf{V}_j$ ，其中 $\tilde{\mathbf{P}}_{ij} \in \mathbb{R}^{B_r \times B_c}, \mathbf{V}_j \in \mathbb{R}^{B_c \times d}$ 。则这里的计算量同样为 $O(B_r B_c d)$

(3) 接下来我们看一共计算了多少次 (1) 和 (2)，也就是执行了多少次内循环：

$$T_c T_r = \frac{N}{B_c} \frac{N}{B_r}$$

(4) 综合以上三点，flash attention的forward计算量为： $O(\frac{N^2}{B_c B_r} B_r B_c d) = O(N^2 d)$ ，注意，因为计算量是用大O阶表示的，所以这里我们把常数项都省略了。

6.3 Flash Attention的显存需求

和标准attention相比，如果不考虑 O 的话，Flash Attention只需要存储 m, l ，其显存需求为 $O(N)$ 。

而标准attention需要存储 S, P ，其显存需求为 $O(N^2)$ 。

可以发现相比于标准attention，flash attention明显降低了对显存的需求。

七、IO复杂度

之前我们强调过，flash attention相比于标准attention的最大优势，就是其减少了对显存（HBM）的访问次数，一定程度上解决了memory bound的问题。所以这一节我们就来具体分析这两者对显存的访问次数（同样都是以forward为例，backward部分论文中也有给出相关推导过程，大家可以类比forward自行阅读）。

7.1 标准attention的IO复杂度

Algorithm 0 Standard Attention Implementation

Require: Matrices $Q, K, V \in \mathbb{R}^{N \times d}$ in HBM.

- 1: Load Q, K by blocks from HBM, compute $S = QK^T$, write S to HBM.
 - 2: Read S from HBM, compute $P = \text{softmax}(S)$, write P to HBM.
 - 3: Load P and V by blocks from HBM, compute $O = PV$, write O to HBM.
 - 4: Return O .
-

知乎 @猛猿

(1) 从HBM中读取 $Q, K \in \mathbb{R}^{N \times d}$ ，计算 $S = QK^T, S \in \mathbb{R}^{N \times N}$ 并将 S 写回HBM。一读一写的IO复杂度为： $O(Nd + N^2)$ ，在表示大O阶时我们忽略常数项。

(2) 从HBM中读取 $S \in \mathbb{R}^{N \times N}$ ，同时计算 $P \in \mathbb{R}^{N \times N}$ 并将其写回HBM。一读一写的IO复杂度为： $O(N^2)$

(3) 从HBM中读取 $P \in \mathbb{R}^{N \times N}, V \in \mathbb{R}^{N \times d}$ ，计算 $O = PV, O \in \mathbb{R}^{N \times d}$ 并将 O 写回HBM。一读一写的IO复杂度为： $O(Nd + N^2)$

所以，总体来说标准attention的IO复杂度为： $O(Nd + N^2)$

7.2 Flash attention的IO复杂度

(1) 我们来看伪代码的第6行，在每个外循环中，我们都会加载 K, V 的block。所有外循环结束后，相当于我们加载了完整的 $K, V \in \mathbb{R}^{N \times d}$ ，因此这里的IO复杂度为： $O(Nd)$

(2) 再看伪代码第8行，在每个内循环中，我们都加载了部分 Q, O, m, l block，由于 m, l 本身比较小(IO复杂度是 $O(N)$)，因此我们暂时忽略它们，只考虑 Q, O （原论文也是这么分析的）。固定某个外循环，所有内循环结束后，我们相当于完整遍历了 $Q, O \in \mathbb{R}^{N \times d}$ 。同时我们会经历 T_c 次外循环。因此这里最终的IO复杂度为： $O(T_c Nd)$ 。

所以，总体来说flash attention的IO复杂度为：

$$O(T_c Nd) = O(\frac{N}{B_c} Nd) = O(\frac{4Nd}{M} Nd) = O(\frac{N^2 d^2}{M})$$

论文中提过，一般d的取值在64~128，M的取值在100KB左右，因此有 $\frac{d^2}{M} \ll 1$ 。因此可以看出，Flash attention的IO复杂度是要显著小于标准attention的IO复杂度的。

八、实验效果

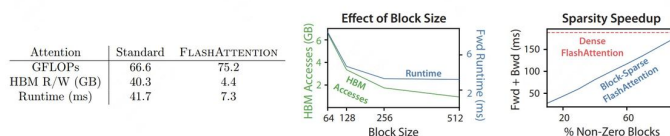


Figure 2: **Left:** Forward + backward runtime of standard attention and FLASHATTENTION for GPT-2 medium (seq. length 1024, head dim. 64, 16 heads, batch size 64) on A100 GPU. HBM access is the primary factor affecting runtime. **Middle:** Forward runtime of FLASHATTENTION (seq. length 1024, head dim. 64, 16 heads, batch size 64) on A100 GPU. Fewer HBM accesses result in faster runtime, up to a point. **Right:** The runtime (for seq. length 1024) of block-sparse FLASHATTENTION is faster than FLASHATTENTION by a factor proportional to the sparsity.

Flash attention的作者将 $N = 1024, d = 64, B = 64$ 的GPT2-medium部署在A100 GPU上，来观测采用flash attention前后的模型的计算性能。

我们先看最左侧图表，标准attention下，计算强度 $I = \frac{66.6}{40.3} = 1.6 < 201$ ，说明GPT2在A100上的训练是受到内存限制的。而在采用flash attention后得到了明显改善，runtime也呈现了显著下降。

我们再来看中间的图表，它表示在使用flash attention的前提下，以forward过程为例，每个数据块的大小对HBM读写次数（绿色）和耗时（蓝色）的影响。可以发现，数据块越大，读写次数越少，而随着读写次数的减少，runtime也整体下降了（复习一下，读写复杂度为 $O(T_c Nd)$ ，数据块越大意味着 T_c 越小）。但有意思的是，当数据块大小>256后，runtime的下降不明显了，这是因为随着矩阵的变大，计算耗时也更大了，会抹平读写节省下来的时间。

九、参考

- 1、FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness
- 2、leimao.github.io/blog/M...
- 3、回旋托马斯x: FlashAttention:加速计算,节省显存, IO感知的精确注意力
- 4、紫气东来: NLP（十七）：从 FlashAttention 到 PagedAttention, 如何进一步优化 Attention 性能
- 5、暖暖内含光: GPU 内存概念浅析
- 6、kaiyuan: GPU内存(显存)的理解与基本使用

大模型

模型加速

GPT

写下你的评论...

80 条评论

默认 最新



榴莲爱喝水

我愿称之为写得最好的教程

2024-06-06 · 新加坡

回复 8



观光居士

期待v2，v2提升更明显，也重写了一部分cuda，但是具体过程还没看懂，太绕了

2023-12-01 · 上海

回复 5



进击的程序猿

1，从 cuda 编程和算法角度理解 SRAM 是 L1 数据缓冲。

flashattention 核心是分块计算注意力嘛，可以简单理解为就是将输入张量划分成很多块，每个数据块放到 sm 里面去计算（cuda/triton 编程的核心就是在于如何将数据分块），sm 里面 L1 cache/共享内存的大小基本就决定了 这个数据块的上限空间大小，所以论文里面说的 SRAM 大小其实值的是 L1 Cache 大小，L2 cache 是很多个 SM 共同能访问的，明显不是论文里指的 SRAM。

2，从 GPU 内存层次角度理解 SRAM 是 L1 数据缓冲。

论文 2.1 节明确都说了 A100 的 SRAM 大小是 192 KB，“As an example, the A100 GPU has 40-80GB of high bandwidth memory (HBM) with bandwidth 1.5-2.0TB/s and 192KB of on-chip SRAM per each of 108 streaming multiprocessors with bandwidth estimated around 19TB/s [44, 45].”

而英伟达官网给出的 A100 白皮书也明确说了 A100 的 L1 cache 大小是 192KB（组合共享内存和 L1 数据缓存），所以论文的 SRAM 肯定指的是 L1 cache 了。

另外，这个论文学习，除非有 cuda 编程经验和 transformer 模型扎实的算法基础就容易理解点，不然真的太难懂啦，很多人其实刚开始就看了个寂寞，而且这个论文也可以说 attention 和 cuda 优化的集大成者啦。

2024-09-30 · 四川

回复 4



RyeWhisky

这里的sram是thread的寄存器

02-19 · 上海

回复 1



istrong

有个问题不太懂，sram是用来做什么的，储存中间数据的结果么，sram这么小能放得下中间数据么，是怎么和hbm配合完成整体数据的传输服务计算的呢

2024-12-30 · 北京

回复 1



OpenLLMAI

来了来了

2023-12-06 · 北京

回复 2



qzylalala

4.2 最后一张图，Q X KT 计算是 [N, d] X [d, N]，得到的 S shape 不应该是 [N, N] 吗，为何图中和上面的描述都是 [N, d]

2024-07-18 · 浙江

回复 3

知乎 图解大模型计算加速系列：FlashAttention V1，从硬件到计算逻辑

切换模式



终于等到大佬更新了

2023-12-06 · 上海

回复 3



离散

唉我记得L2 Cache才是shared memory，L1 Cache对程序员是透明的，用户存放寄存器的；这方面你是参考了哪里呀？我看你提到了L1 才是SMEM

2023-12-02 · 浙江

回复 2



zzk again

VOLTA架构后，L1和shared memory合并到一起了，可以通过一些configuration来分配L1的多少比例给shared memory

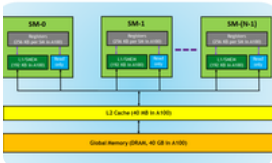
2023-12-03 · 广东

回复 7



离散 · 猛猿

确实是我一开始理解错了（看了stackoverflow好像还有一个哥们和我一样理解错了，真是神奇）看这张图里L1和Shared其实不是一个东西，但是他们是在一块的。
[CUDA Refresher: The CUDA Programming Model | NVIDIA Technical Blog](#)
另外L1 和 shared的差别参考 shared mem是交给用户控制的，L1是用户不透明的，[What's the difference between L1 cache and the shared memory](#)



2023-12-02 · 浙江

回复 2

展开其他 3 条回复



翠竹叶飞

backward中的计算量感觉文章Appendix里给的 $O(N^2)$ 应该是少乘了 d ，这个应该没法省略lol

05-06 · 天津

回复 1



k9sret · Don Fellowshawn

d 指的是每个头的head dim，一般64、128或256，4096是多个头加起来

08-01 · 浙江

回复 喜欢



Don Fellowshawn

支持，一般你 N 搞个 $32k=32768$ ，算是长上下文了， d 一般是4096，这可以说是同一个量级的，根本不能忽略

05-10 · 上海

回复 喜欢



无才

写的非常好。有个小问题，“在GPT类的模型中，还需要对P做mask处理”这里的表述是不是不太对，attention计算中应该是对S进行mask（赋负极大值），在进行softmax

02-07 · 福建

回复 1

知乎 图解大模型计算加速系列：FlashAttention V1，从硬件到计算逻辑

切换模式

如果Q K V很小，是个mem bound的，是有机会把Q K V全部加在到GPU内部，依次算出S P O，最后统一写到HBM。从这个角度看，有没有flashattention区别不大。

如果Q K V比较大，GPU内部L2/L1放不下，没有flashattention，只能先算S，再算P，再算O。因为L2/L1放不下，这个过程也只能分tile计算。此时flashattention就派上用场了，分tile不是他的创新，他的创新是分tile基础上把算子融合了，每个tile依次完成S P O。减少了HBM访问的次数。从这个角度看，存储他并没有节省，因为本来人家就要分tile解决L2/L1不够的问题。

是不是整个逻辑变成了，flashattention的优势是提供了一种算子融合的方案。

2024-07-11 · 四川

回复 1

id17 点击查看全部评论 > 分tile并且更新m和l是flashattention的创新吧？ IO复杂度下降了； 存S和P变成存m

写下你的评论...

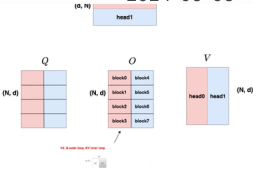
一心搞钱的韭菜 以前CPU时期的算法是用空间换时间（其实是内存换计算），典型的是动态规划。

推荐阅读

这个flash attention更像是计算换内存和IO， forward阶段增加的不明显，量级上增加的是O(TrNd)，要比总计算量级O(dN^2)小很多， backward阶段重新计算S和P（主要是S）这个计算量是2dN^2，在总计算量面前实际并不能被忽略。

2024-09-05 · 浙江

回复 喜欢



图解大模型计算加速系列：Flash Attention V2，从原理...

猛猿

仿真加速-01 如何加速 Simulink 执行阶段

2025-06-13 增加 4 给模型引用分配合理的求解器 2025-06-12 更新增加 3 处理代数环 2025-06-10 更新部分图片。内容作了调整，初始化和编译阶段分做其它篇。 2024-09-22 更新部分手机上看不...

基于模型设计柚子



Python实现GPU加速的基本操作

酷番云



FlashAttention对GQA的处理方法及其v2包含Causal maskin...

三好学生亚可